# Лекция №2

## Основы веб-технологий

Дисциплина: «Информационные технологии и программирование»

# Вопросы лекции

1. Модель клиент–сервер
2. Сетевые протоколы и уровневая модель сети
3. Как браузер получает веб-страницу
4. HTML: структура документа и основные элементы
5. CSS: селекторы, каскад, блочная модель
6. Flexbox, Grid и адаптивность
7. Типичные ошибки и проверка страницы

# Модель клиент–сервер

- **Клиент** — программа, запрашивающая ресурс (браузер)
- **Сервер** — компьютер, хранящий и отдающий ресурсы
- **Ресурс** — HTML-страница, CSS, изображение, данные JSON
- Схема взаимодействия: **запрос** → **ответ**

# Что такое сетевой протокол

**Сетевой протокол** — набор правил, определяющих формат и порядок обмена данными между устройствами.

Протокол описывает:

- как обратиться к другому устройству;
- как передаются данные;
- как обрабатываются ошибки и потери;
- как завершается обмен.

# Уровневая модель сети

**Модель OSI (7 уровней):**
физический → канальный → сетевой → транспортный → сеансовый → представительский → прикладной

**Модель TCP/IP (практический взгляд, 4 уровня):**
сетевой доступ → межсетевой → транспортный → прикладной

# Уровни и протоколы в вебе

| Уровень | Назначение | Протоколы |
|---|---|---|
| Прикладной | данные между приложениями | HTTP, HTTPS, DNS |
| Транспортный | доставка между процессами | TCP, UDP |
| Межсетевой | адресация и маршрутизация | IP |
| Сетевой доступ | передача внутри локальной сети | Ethernet, Wi-Fi |

# IP-адресация и DNS

- **IP-адрес** — числовой адрес устройства в сети: `93.184.216.34`
- Человеку неудобно запоминать числа → существуют доменные имена
- **DNS** (Domain Name System) преобразует имя в адрес:
  `example.com → 93.184.216.34`
- Без работающего DNS сайт недоступен по имени, даже если сервер исправен

# Протоколы TCP и UDP

**TCP** — надёжная доставка:

- установка соединения;
- разбивка данных на пакеты и контроль порядка;
- повторная передача потерь;
- подтверждение получения.

**UDP** — быстрая доставка без гарантий (видео, аудио, игры).

Веб-страницы передаются по **TCP**: код должен прийти целиком и без искажений.

# Протокол HTTP

**HTTP** — HyperText Transfer Protocol, протокол передачи гипертекста.

Запрос браузера:

```
GET /index.html HTTP/1.1
Host: example.com
```

Ответ сервера:

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
```

# Методы HTTP и коды состояния

**Методы:** GET, POST, PUT, DELETE, HEAD, OPTIONS

**Коды состояния ответа:**

- `200 OK` — успех
- `301 / 302` — перенаправление
- `403 Forbidden` — доступ запрещён
- `404 Not Found` — ресурс не найден
- `500 Internal Server Error` — ошибка сервера

# HTTPS и защита соединения

**HTTPS** = HTTP поверх защищённого соединения (TLS).

Обеспечивает:

- шифрование передаваемых данных;
- защиту от подмены и перехвата;
- проверку подлинности сервера.

Обязателен, если страница принимает пароли, личные данные, платежи.

# Как браузер получает страницу

1. Пользователь вводит `https://example.com/index.html`
2. DNS: имя → IP-адрес сервера
3. Установка TCP-соединения (плюс TLS при HTTPS)
4. HTTP-запрос `GET /index.html`
5. Сервер возвращает HTML-документ
6. Браузер находит в нём `<link>` и `<img>` → дозапрашивает CSS и изображения
7. Страница строится и отображается пользователю

# HTML: язык гипертекстовой разметки

**HTML** — HyperText Markup Language.

- Описывает **структуру и смысл** содержимого, но не внешний вид
- Элемент = тег + атрибуты + содержимое: `<p>`, `<a href="...">`
- Гипертекст = текст со ссылками на другие документы
- За внешний вид отвечает CSS

# Структура HTML-документа

```
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Заголовок вкладки</title>
</head>
<body>
    <h1>Заголовок страницы</h1>
    <p>Видимое содержимое.</p>
</body>
</html>
```

# Головная часть `head`

- `<meta charset="UTF-8">` — кодировка; без неё кириллица превращается в «кракозябры»
- `<meta name="viewport" ...>` — корректный масштаб на мобильных устройствах
- `<title>` — заголовок вкладки и сниппет в поиске
- `<link rel="stylesheet" href="css/style.css">` — подключение внешнего файла стилей

# Текст: заголовки и абзацы

```
<h1>Один на страницу</h1>
<h2>Раздел</h2>
<h3>Подраздел</h3>
<p>Абзац текста.</p>
<strong>Важно</strong> и <em>акцент</em>
```

- Уровни заголовков отражают **логическую структуру** документа
- Уровень не выбирают «ради размера шрифта» — размер настраивает CSS

# Гиперссылки

```
<a href="https://example.com">Внешний ресурс</a>
<a href="pages/about.html">Относительный адрес</a>
<a href="#about">К разделу страницы</a>
```

- Абсолютный адрес — с протоколом и доменом; относительный — от текущего документа
- Внутренняя ссылка работает в паре с атрибутом `id` целевого элемента

# Списки

```
<ul>            <!-- маркированный -->
    <li>Пункт</li>
</ul>

<ol>            <!-- нумерованный -->
    <li>Шаг</li>
</ol>
```

- `ul` — порядок элементов не важен
- `ol` — важна последовательность
- Списки могут быть вложенными; меню сайта — тоже список ссылок

# Изображения

```
<img src="images/image.jpg"
     alt="Описание изображения"
     width="600">
```

- `src` — путь к файлу, `width` — ширина
- `alt` — альтернативный текст: отображается при ошибке загрузки, читается скринридерами
- Пустой `alt` — ошибка; в практической работе он обязателен

# Таблицы

```
<table>
    <tr><th>Название</th><th>Значение</th></tr>
    <tr><td>Данные</td><td>Данные</td></tr>
</table>
```

- `tr` — строка, `th` — заголовочная ячейка, `td` — ячейка данных
- Таблица — для **табличных данных**, а не для раскладки страницы

# Семантические элементы страницы

```
<header>  <nav>  <main>
<section> <article> <aside> <footer>
```

- Делают структуру страницы понятной людям и машинам
- Улучшают доступность и индексирование поисковыми системами
- Упрощают поддержку кода и подключение стилей

# CSS и способы подключения

**CSS** — Cascading Style Sheets: внешний вид и расположение элементов.

Способы подключения:

1. Внешний файл: `<link rel="stylesheet" href="css/style.css">` — основной способ
2. Тег `<style>` в головной части
3. Атрибут `style` у элемента — только в исключительных случаях

# Селекторы и синтаксис правил

Правило = селектор + блок объявлений `{ свойство: значение; }`

```
p           { color: #333333; }
.card       { padding: 20px; }
#about      { margin-top: 40px; }
.main-nav a { text-decoration: none; }
h1, h2, h3  { font-family: Arial, sans-serif; }
```

# Каскад, наследование, специфичность

- **Каскад**: при конфликте правил побеждает более важное, специфичное, а затем более позднее
- **Специфичность**: тег < класс < идентификатор < inline-стили
- **Наследование**: `font-family` и `color` наследуются потомками
- Пример: `p { color: black; }` и `.text { color: gray; }` → победит класс

# Блочная модель документа

Каждый элемент — прямоугольник: содержимое → padding → border → margin

```
.card {
    width: 300px;
    padding: 20px;
    border: 1px solid #dddddd;
    margin: 10px;
}
* { box-sizing: border-box; }
```

# Раскладка: Flexbox

```
.main-nav ul {
    display: flex;
    flex-wrap: wrap;
    gap: 16px;
}
```

- Одномерная раскладка: строка **или** столбец
- `justify-content` — вдоль главной оси, `align-items` — вдоль поперечной
- Идеален для меню и рядов однотипных элементов

# Раскладка: Grid

```
.cards {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 20px;
}
```

- Двумерная сетка: строки и столбцы одновременно
- `1fr` — доля свободного пространства колонки
- Flexbox — лента; Grid — сетка карточек и сложные макеты

# Адаптивность и медиазапросы

```
@media (max-width: 768px) {
    .main-nav ul { flex-direction: column; }
    .cards { grid-template-columns: 1fr; }
}
```

- Одну страницу смотрят на мониторе, ноутбуке и телефоне
- Точки перестроения: 900px, 768px, 600px
- На мобильных: вертикальное меню, карточки в одну колонку, без горизонтальной прокрутки

# Проект и проверка результата

```
web-page/
├── index.html
├── css/style.css
└── images/image.jpg
```

Проверка:

- валидаторы W3C для HTML и CSS;
- инструменты разработчика: структура, стили, эмуляция ширины;
- вручную: ссылки, списки, изображение, таблица, карточки, меню.

# Типичные ошибки

**HTML:**

- отсутствуют `<!DOCTYPE html>`, `lang`, `charset`, `viewport`;
- несколько `h1`; пустой `alt`; таблица без заголовков `th`.

**CSS и раскладка:**

- не подключён файл стилей; нет `box-sizing: border-box`;
- меню не переносится; карточки не перестраиваются;
- горизонтальная прокрутка на мобильной ширине.

# Итоги лекции

- Веб работает по модели клиент–сервер поверх стека протоколов
- Путь страницы: DNS → IP → TCP → HTTP(S) → HTML и CSS в браузере
- HTML отвечает за структуру и смысл, CSS — за внешний вид и раскладку
- Flexbox и Grid — современные инструменты раскладки
- Медиазапросы обеспечивают адаптивность на разных экранах